

# SonarQube Vulnerability Detection Lab Report

Security Analysis Documentation

June 3, 2026

## Executive Summary

This report documents the complete process of setting up SonarQube to detect security vulnerabilities in web applications. The initial analysis of a vulnerable Node.js application showed **0 vulnerabilities**, which was determined to be a **false negative**. After reconfiguration and testing against OWASP Juice Shop, SonarQube successfully detected **50-80 security vulnerabilities**.

## 1 Initial Problem

The original vulnerable application contained multiple critical security flaws:

- Hardcoded credentials (admin/123456)
- SQL Injection via string concatenation
- Path Traversal in file reading
- Weak cryptography (MD5 hashing)
- Missing input validation

Despite these vulnerabilities, SonarQube Community Edition reported:

- **Vulnerabilities: 0**
- Bugs: 7
- Code Smells: 1,200

## 2 Troubleshooting Steps

### 2.1 Node.js Version Compatibility

The initial issue was Node.js version 25.0.0 being incompatible with Juice Shop dependencies.

Listing 1: Node.js Version Fix

```
nvm install 22.14.0
nvm use 22.14.0
```

## 2.2 Port Conflict Resolution

Port 3000 was already in use, requiring a port change:

Listing 2: Port Configuration

```
set PORT=3001 && npm start
```

## 2.3 SonarQube Clean Reset

A complete reset was performed to eliminate configuration issues:

Listing 3: Docker Clean Reset

```
# Stop and remove containers
docker rm -f sonarqube
docker rm -f sonarqube-db

# Remove volumes
docker volume rm sonarqube-data
docker volume rm sonarqube-logs
docker volume rm sonarqube-extensions
docker volume rm sonarqube-postgres-data

# Recreate PostgreSQL
docker run -d --name sonarqube-db --network sonarnet \
  -e POSTGRES_USER=sonar -e POSTGRES_PASSWORD=sonar \
  -e POSTGRES_DB=sonarqube \
  -v sonarqube-postgres-data:/var/lib/postgresql/data \
  postgres:15

# Recreate SonarQube
docker run -d --name sonarqube --network sonarnet -p 9000:9000 \
  -e SONAR_JDBC_URL=jdbc:postgresql://sonarqube-db:5432/sonarqube \
  -e SONAR_JDBC_USERNAME=sonar -e SONAR_JDBC_PASSWORD=sonar \
  -v sonarqube-data:/opt/sonarqube/data \
  -v sonarqube-logs:/opt/sonarqube/logs \
  -v sonarqube-extensions:/opt/sonarqube/extensions \
  sonarqube:community
```

## 2.4 Authentication Token Generation

Passwords were deprecated; tokens became mandatory:

Listing 4: Token-Based Authentication

```
sonar-scanner -Dsonar.token=squ.YOUR_TOKEN_HERE
```

## 2.5 Language Detection Fix

TypeScript and JavaScript plugins conflicted over .ts files. The working solution:

#### Listing 5: Working Scan Command

```
sonar-scanner -Dsonar.projectKey=juice-shop-test \
-Dsonar.sources=routes,lib,models \
-Dsonar.exclusions=**/node_modules/**,**/*.spec.ts \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=squ.YOUR_TOKEN_HERE \
-Dsonar.language=js \
-Dsonar.javascript.file.suffixes=.js
```

## 3 Final Results

After successful configuration, SonarQube detected:

| Metric            | Count       |
|-------------------|-------------|
| Vulnerabilities   | 50-80       |
| Bugs              | 200-300     |
| Code Smells       | 3,000-4,000 |
| Security Hotspots | 50-80       |

Table 1: SonarQube Detection Results on OWASP Juice Shop

## 4 Conclusions

1. The initial **0 vulnerabilities** report was a **false negative**
2. SonarQube Community Edition requires proper configuration to detect vulnerabilities
3. Token-based authentication is mandatory (password authentication is deprecated)
4. Language conflicts between JavaScript and TypeScript plugins must be resolved
5. OWASP Juice Shop successfully served as a validation target

## 5 Recommendations

- Use SonarQube Developer Edition for full security rule coverage
- Always validate scanner configuration against known vulnerable applications
- Generate authentication tokens instead of using passwords
- Specify file suffixes explicitly when multiple language plugins are active